

Technical Memo — Indian License Plate Detection API

Candidate Submission | RT-DETR Object Detection + Reasoning Layer

1. Domain & Dataset Choice

I chose vehicle license plate detection as the problem domain, motivated by a practical use case: automated parking entry/exit logging for apartments, campuses, and toll systems — without expensive proprietary hardware.

Dataset: Sourced from Roboflow Universe ([ross-gvxki/number-plates-plbas](#), version 2, CC BY 4.0). The dataset contains approximately 1,490 images of vehicles with annotated license plate bounding boxes under a single class: [license-plates](#). This class is explicitly absent from the standard COCO class set (which contains generic object categories but no license plate class), satisfying the non-COCO constraint.

Why this dataset: It contains diverse vehicle types, lighting conditions, and plate positions — making it a realistic proxy for real-world deployment. Labels were verified by spot-checking 30 random samples visually before training.

2. Train / Val / Test Split

Used the dataset's provided split:

- **Train:** ~70% of images
- **Val:** ~20% of images
- **Test:** ~10% of images

The test set was kept completely untouched until final evaluation — no hyperparameters were tuned against it. Validation set was used only for monitoring loss curves during training. This prevents data leakage and gives an honest estimate of generalization.

3. Evaluation Metrics & What They Tell Us

Metric	Value
mAP@50	0.995
mAP@50-95	0.924

Precision	0.996
Recall	1.000

What these mean:

- **mAP@50 (0.995):** The model localizes and classifies license plates nearly perfectly at IoU threshold 0.5 — meaning predicted boxes overlap ground truth by at least 50%.
- **mAP@50-95 (0.924):** At stricter localization thresholds, performance drops slightly — indicating the model occasionally produces boxes that are slightly imprecise at the pixel level.
- **Precision (0.996):** Almost no false positives — when the model says it found a plate, it almost always has.
- **Recall (1.0):** The model misses zero plates in the test set.

What these don't tell us:

- Performance on night-time or heavily occluded plates
- Generalization to Indian-specific plates (dataset is primarily UK/European plates)
- Real-time latency under concurrent API load
- Performance on low-resolution CCTV footage

4. Five Failure Cases & Root Cause Analysis

These were identified by running inference on edge-case images outside the test set:

1. Extreme angle / perspective distortion

Plates photographed from steep angles ($>45^\circ$) cause the model to miss or produce loose bounding boxes. Root cause: training data is predominantly front/rear-facing shots with minimal perspective variation.

2. Partially occluded plates

When a plate is $>50\%$ covered (by a tow bar, sticker, or dirt), the model either misses it entirely or produces a low-confidence detection. Root cause: occlusion augmentation was not explicitly applied during training.

3. Night-time / low-light images

Under poor lighting, plate visibility drops and model confidence falls below the 0.3 threshold. Root cause: dataset is biased toward daytime, well-lit images.

4. Very small plates in wide-angle shots

Plates occupying fewer than 20×10 pixels in a 640×640 input are frequently missed. Root cause: RT-DETR's attention mechanism struggles with very small objects at standard input resolution.

5. Non-standard plate formats

Temporary plates, dealer plates, or heavily customized fonts produce false negatives. Root cause: training data does not represent these edge cases, causing the model to not recognize them as license plates.

5. Part B Reasoning Layer Design

The reasoning layer is a **single hand-written decision function** — no LangChain, LangGraph, or agentic frameworks used anywhere.

Intent Router:

A keyword list (`plate`, `number`, `license`, `vehicle`, `how many`, `count`, `visible`, `registration`, `identify`) determines whether the question requires calling the detector. If no keyword matches, the system responds immediately without running inference — saving compute and giving an honest "this question isn't about the image" response.

Confidence Guardrail:

After detection, two checks run before answering:

1. If `len(detections) == 0` → returns *"Insufficient information: No license plates detected with sufficient confidence."*
2. If `avg_confidence < 0.4` → returns *"Insufficient information: Detection confidence too low to answer reliably."*

Example where system correctly says insufficient information:

- Input: A dark, blurry image of a parking lot at night
- Question: *"How many vehicles have visible plates?"*
- Model returns 0 detections above 0.3 threshold
- Response: *"Insufficient information: No license plates were detected with sufficient confidence. The image may be unclear, too dark, or no plates are visible."*

This design ensures the API never hallucinates an answer when the visual evidence is weak.

Hardware & Reproducibility

Parameter	Value
Hardware	Google Colab T4 GPU (16GB)
Training time	~60 minutes

Epochs	30
Batch size	8
Image size	640×640
Learning rate	0.0001
Warmup epochs	3
Weight decay	0.0001
Base model	RT-DETR-L (rtdetr-l.pt)
Framework	Ultralytics 8.4.150